

Lab Exercise 5: Frontend Angular Project

For exercises 2-5, create a blogging site using ASP.NET Web API and SQL Server Database. The site must have log in, registration, and post functionality (adding a post, listing all posts, and showing details of one post).

This specific lab exercise will deal with UI and frontend functionality, using Angular and Typescript.

The following should be downloaded:

- Visual Studio Code
- Node.Js
Download Node version 16 from <https://nodejs.org/en/blog/release/v16.16.0/> Verify whether the installation was successful by going to Command Prompt/Powershell and type `node -v`
- Angular
In VS Code, click Terminal > new Terminal. After installing Node and verifying it was successful, install Angular by typing `npm install @angular/cli`

Setting up the project

1. Create a new folder named <LASTNAME>LE4.
2. Open the folder in VS Code.
3. Create a new terminal in VS Code.
4. Create a new project by typing `ng new frontend`
5. Change directory in the terminal by typing `cd frontend`
6. To run the project, type in `ng serve --open`

Create a new component

1. In the terminal, to generate a component, type in `ng generate component components/list-posts`
2. There should be four new files under components/list-posts folder.

Setting up models

1. In the terminal, type in `ng generate class models/post --type=model`
2. There should be a new class under the models folder named `post.model.ts`.
3. Type in the following code inside the file:

```
export type Post = {  
  id: number;  
  title: string;  
  body: string;  
  dateCreated: Date;  
  userName: string;  
  firstName: string;  
  lastName: string;  
}
```

Make sure to use `export type Post` in the first line, not `class`.

Getting data from the API

1. Inside the `list-posts.component.ts`, create a variable called `posts` to hold the data.
It is usually placed above the constructor, right below the `export class` line.

```
posts?: Post[] = [];
```

Since there may be no data returned from the API (if there are no posts in the database, or there is an error), the question marks allows the variable to be null.

2. In the constructor, add a private variable `http`, and importing `HttpClient` as seen in the following:



```
constructor(private http: HttpClient) { }
```

3. Create a new method called `initData()` and call it in `ngOnInit()` adding the following lines of code:



```
ngOnInit(): void {  
  this.initData();  
}  
  
initData(): void {  
  this.http.get<Post[]>('https://localhost:7161/api/post')  
    .subscribe({  
      next: (data: Post[]) => {  
        this.posts = data;  
        console.log(this.posts);  
      }  
    })  
}
```

The localhost port number may be different.

4. In `app-routing.module.ts`, add the following line in the routes:

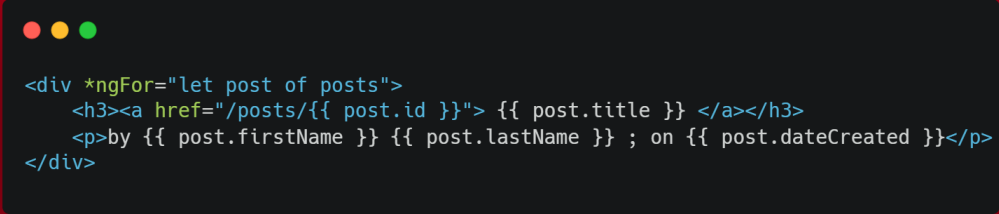


```
const routes: Routes = [
  {path: '', component: ListPostsComponent},
];
```

5. Run the backend made last lab exercise.
6. Run the frontend by typing `ng serve -open` in the terminal
7. The page should be blank. However, if you type in Ctrl+Shift+J, it should open the Developer Tools.
8. The console should contain the data from the backend.

Configuring HTML files

1. Open the list-posts.component.html.
2. Type in the following code:



```
<div *ngFor="let post of posts">
  <h3><a href="/posts/{{ post.id }}"> {{ post.title }} </a></h3>
  <p>by {{ post.firstName }} {{ post.lastName }} ; on {{ post.dateCreated }}</p>
</div>
```

*This is an equivalent of the for loop in angular. *ngFor takes the posts variable in the typescript file.*

3. Save the file. Angular reloads the page whenever there are pages. The data from the API should now show.
4. Screenshot the page.

Showing post details

1. Generate a new component named post-detail by typing `ng g c components/post-detail`
Type in Ctrl+C to terminate the running application. The command is also a shorthand for generate and component keywords.
2. In post-detail.component.ts, initiate the following variables:



```
private routeSub: Subscription = new Subscription();  
private id: number = 0;  
  
post?: Post;
```

3. In the constructor parameters, add the following modules:



```
constructor(  
    private route: ActivatedRoute,  
    private http: HttpClient  
) { }
```

4. Inside ngOnInit, add the following code:

```
ngOnInit(): void {
  this.routeSub = this.route.params.subscribe(params => {
    this.id = params['id'];
  })
  this.initData();
}
```

This would take the ID based off of the URL of the page.

5. Create the method `initData()`. Put in the code below to get the data from the API.

```
initData(): void {
  this.http.get<Post>("https://localhost:7161/api/post/" + this.id).subscribe({
    next: (data: Post) => {
      this.post = data;
      console.log(this.post);
    }
  })
}
```

6. In `app-routing.module.ts`, add the following line:

```
const routes: Routes = [
  {path: '', component: ListPostsComponent},
  {path: 'posts/:id', component: PostDetailComponent},
];
```

7. Run the backend, and the frontend.
8. In the main page, click on the title of the first post you ever made.
9. The page should be blank. Open the console and the data should be logged.
10. In `post-detail.component.html`, add the following code:

```

<h3><a href="/posts/{{ post?.id }}"> {{ post?.title }} </a></h3>
<p>by {{ post?.firstName }} {{ post?.lastName }}; on {{ post?.dateCreated }}</p>
<p>{{ post?.body }}</p>

```

11. Save the file and go back to the browser. Screenshot the page.

Setting up authentication

1. In VS Code Terminal, create a new component called login-page under the components folder.
2. In the HTML file, code the following:

```

<div>
  <h1>Welcome Back</h1>
  <h3>Login Here</h3>
  <form name="form" (ngSubmit)="onSubmit()" #f="ngForm" novalidate>
    <div>
      <label for="username">Email</label>
      <input
        required
        type="text"
        name="username"
        [(ngModel)]="form.username"
        #username="ngModel">

      <label for="password">Password</label>
      <input type="password"
        name="password"
        [(ngModel)]="form.password"
        required
        #password="ngModel">
    </div>
    <button>Submit</button>
  </form>
</div>

```

3. In the terminal, create a new service called token-storage service under the services folder. Use the command `ng generate services/token-storage service`
4. In the token-storage.service.ts, include the following code:

```

constructor() { }

signOut(): void {
    window.sessionStorage.clear();
}

public saveToken(token: string): void {
    window.sessionStorage.removeItem(TOKEN_KEY);
    window.sessionStorage.setItem(TOKEN_KEY, token);
}

public getToken(): string | null {
    return window.sessionStorage.getItem(TOKEN_KEY);
}

public saveUser(id : number): void {
    let strId = id.toString();
    window.sessionStorage.removeItem(USER_KEY);
    window.sessionStorage.setItem(USER_KEY, strId);
}

```

5. Create another service called auth under the services folder.
6. Put in the following code inside the auth.service.ts:

```

export class AuthServiceService {
    isLoggedIn: boolean = false;
    public redirectUrl: string = "";

    constructor(private http: HttpClient) {
    }

    login (username: string, password: string) {
        return this.http.post<string>("https://localhost:7161/api/login/login", {username, password});
    }
}

```

7. Add the following code under the login-page.component.ts.

```

export class LoginPageComponent implements OnInit {
  form: any = {
    username: null,
    password: null
  }

  constructor(
    private authService: AuthServiceService,
    private tokenStorage: TokenStorageService,
    private http: HttpClient,
    private router: Router) { }

  ngOnInit(): void {
    if (this.tokenStorage.getToken()) {
      this.authService.isLoggedIn = true;
      this.router.navigate([this.authService.redirectUrl]);
    }
  }

  onSubmit() {
    const {username, password} = this.form;

    this.http.post<LoginPostData>("https://localhost:7161/api/Login/login", { username,
    password }).subscribe(data => {
      this.tokenStorage.saveToken(data.id_token);
      this.tokenStorage.saveUser(data.id);
      this.router.navigate([this.authService.redirectUrl]);
      window.location.reload();
    })
  }
}

export interface LoginPostData {
  id_token: string;
  id: number;
}

```

8. Lastly, add a new component named register-page.component.ts under components folder.
9. In the HTML file, include the following code:

```
<form
  name="form"
  (ngSubmit)="onSubmit()"
  #f="ngForm">

  <label for="username">Username</label>
  <input
    type="text"
    name="username"
    [(ngModel)]="form.username"
    required
    #username="ngModel" />

  <label for="password">Password</label>
  <input
    type="password"
    name="password"
    [(ngModel)]="form.password"
    required
    #password="ngModel" />

  <label for="firstName">First Name</label>
  <input
    type="text"
    name="firstName"
    [(ngModel)]="form.firstName"
    required
    #firstName="ngModel" />

  <label for="lastName">Last Name</label>
  <input
    type="text"
    name="lastName"
    [(ngModel)]="form.lastName"
    required
    #lastName="ngModel" />

  <button>Submit</button>

</form>
```

10. Then add the following code to the TypeScript file:

```
export class RegisterPageComponent implements OnInit {
  form: any = {
    username: null,
    password: null,
    firstName: null,
    lastName: null
  }

  constructor(private http: HttpClient,
    private route: Router) { }

  ngOnInit(): void {
  }

  onSubmit(): void {
    const {
      username, password, firstName, lastName
    } = this.form

    console.log(this.form);

    this.http.post("https://localhost:7161/api/Login/register", this.form, {responseType:
'text'}).subscribe(data => {
      this.route.navigate(['/login'])
    })
  }
}
```

Post-lab requirements:

A.

1. Screenshot the entire project folder path from your IDE. Make sure to include folders that has your name.
2. Push your code to GitHub. Paste the link at the end of the document.
3. **Put all screenshots and codes of this activity in a MS Word file and submit it in the LE5-1 Blackboard link provided. Filename should be LE5-1-SURNAME**

B. Run the program.

1. Create an MP4 video demonstrating this laboratory exercise 5. **Submit the MP4 video in the LE5-VID Blackboard link provided. Filename should be LE5-VID-SURNAME.**